DOCUMENTATION MOBILE

Projet EvenixMobile

React Native · Expo · AsyncStorage · JWT

BTS SIO SLAM — 2025-2026

Table des matières

1. Introduction
2. Architecture du projet
3. Technologies utilisées
4. Fonctionnalités
5. Code par fonctionnalité
6. Problèmes rencontrés et solutions
7. Tests et validation

1. Introduction

EvenixMobile est une application mobile développée en React Native avec Expo. Elle se connecte à une API REST Spring Boot pour permettre à l'utilisateur de consulter des événements et de faire des réservations sécurisées par token JWT.

Objectifs

- Créer une interface mobile native (Android / iOS) testable via Expo Go
- Authentifier l'utilisateur avec JWT, stocker le token dans AsyncStorage
- Afficher la liste des événements et le détail de chacun
- Envoyer une réservation POST avec le header Authorization: Bearer <token>
- Gérer proprement les codes HTTP : 200, 400, 401, 403 et erreurs réseau

Architecture générale

L'application est organisée en 3 écrans (screens) connectés par une navigation Stack. Un fichier services/api.js centralise l'URL du backend. AsyncStorage remplace le localStorage du web pour la persistance locale.

2. Architecture du projet

Structure des fichiers

```
EvenixMobile/
├── App.js                      ← Point d'entrée, charge AppNavigator
├── navigation/
│   └── AppNavigator.js         ← Stack Navigator (3 écrans)
├── screens/
│   ├── LoginScreen.js          ← Authentification JWT
│   ├── EventsListScreen.js     ← Liste des événements (FlatList)
│   └── EventDetailScreen.js    ← Détail + bouton Participer (POST)
        FavoritesScreen.js      ← pour mettre en favorise un évènement
├── services/
│   └── api.js                  ← BASE_URL du backend
        FavorisService.js
└── package.json
```

Couches de l'application

Couche	Fichier	Rôle
Configuration	services/api.js	Exporte l'IP du backend (pas localhost)
Navigation	navigation/AppNavigator.js	Stack.Navigator + 3 écrans
Auth	screens/LoginScreen.js	POST /login → JWT → AsyncStorage
Liste	screens/EventsListScreen.js	GET /evenements/all → FlatList
Détail + Réservation Détail + Favorite	screens/EventDetailScreen.js screens/FavoritesScreen.js	GET getById + POST save (Bearer) GET /evenements/all → Favorise

Flux de navigation

LoginScreen POST /api/auth/login	**replace()** ⟶	**EventsList** GET /evenements/all	**navigate(id)** ⟶	**EventDetail** GET getById/{id} POST reservations/save

3. Technologies utilisées

Technologie	Version	Usage
React Native + Expo	SDK 51	Framework mobile iOS / Android
Expo Go	—	Application de test sur smartphone physique
React Navigation (Native Stack)	6.x	Navigation entre les 3 écrans
AsyncStorage	1.x	Stockage local : token, userId, email, roles
fetch() natif	—	Appels GET et POST vers le backend Spring Boot
JWT (Bearer token)	—	Sécurisation des requêtes POST authentifiées
Visual Studio Code	—	IDE principal
Postman	—	Tests des endpoints avant intégration
GitHub	—	Versioning du code source

Commandes d'installation

```
# Créer le projet
npx create-expo-app@latest EvenixMobile --template blank

# React Navigation
npx expo install @react-navigation/native @react-navigation/native-stack
npx expo install react-native-screens react-native-safe-area-context

# AsyncStorage
npx expo install @react-native-async-storage/async-storage

# Lancer (--tunnel si réseau scolaire/entreprise)
npx expo start --tunnel
```

4. Fonctionnalités — TP2 à TP7

	Fichier(s)	Ce qui a été fait
	services/api.js	Création du projet Expo + configuration BASE_URL

	screens/LoginScreen.js	Formulaire email/mot de passe + POST + AsyncStorage
	navigation/AppNavigator.js, App.js	Stack Navigator + replace() après login
	screens/EventsListScreen.js	FlatList + useEffect + gestion chargement/erreur
	screens/EventDetailScreen.js	ScrollView + sections + navigation.setOptions()
	screens/EventDetailScreen.js	Bouton Participer + POST JWT + codes HTTP 401/403
	screens/FavoritesScreen.js	Bouton Favorise + qui permet de mettre en favorites un événement

Détail des fonctionnalités clés

Bouton Participer (fonctionnalité principale)
Lecture du token depuis AsyncStorage, envoi en header Authorization: Bearer <token>. Gestion des réponses : 200 → confirmation, 401 → token expiré + retour Login, 403 → rôle insuffisant, erreur réseau → message dédié. Le bouton affiche un spinner et est désactivé pendant l'envoi pour éviter le double-clic.

Navigation — replace() vs navigate()
replace() au lieu de navigate() après le login — l'utilisateur ne peut pas revenir à l'écran Login avec le bouton retour. La pile de navigation est remplacée, pas empilée.

Performance — FlatList vs .map()
FlatList ne rend que les éléments visibles à l'écran, contrairement à .map() qui génère tout en mémoire. Indispensable pour les longues listes d'événements.

5. Code par fonctionnalité

services/api.js

L'IP du PC est obligatoire (pas localhost) — sur smartphone, localhost pointe vers le téléphone lui-même.

```
const BASE_URL = 'http://192.168.X.X:8081';  // IP de votre PC
export default BASE_URL;
```

navigation/AppNavigator.js

```
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import LoginScreen from '../screens/LoginScreen';
import EventsListScreen from '../screens/EventsListScreen';
import EventDetailScreen from '../screens/EventDetailScreen';

const Stack = createNativeStackNavigator();

export default function AppNavigator() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName='Login'>
        <Stack.Screen name='Login' component={LoginScreen}
          options={{ headerShown: false }} />
        <Stack.Screen name='EventsList' component={EventsListScreen}
          options={{ title: 'Événements' }} />
        <Stack.Screen name='EventDetail' component={EventDetailScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

LoginScreen.js — Connexion JWT

```
const handleLogin = async () => {
  setLoading(true);
  try {
    const response = await fetch(`${BASE_URL}/api/auth/login`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password }),
    });
    if (!response.ok) throw new Error('Identifiants incorrects');
    const data = await response.json();
    await AsyncStorage.setItem('token', data.token);
    await AsyncStorage.setItem('userId', String(data.userId));
    navigation.replace('EventsList');
  } catch (err) {
    Alert.alert('Erreur', err.message);
  } finally {
    setLoading(false);
  }
};
```

EventsListScreen.js — Liste avec FlatList

```
useEffect(() => { chargerEvenements(); }, []);

const chargerEvenements = async () => {
  setLoading(true);
  try {
    const response = await fetch(`${BASE_URL}/api/evenements/all`);
    if (!response.ok) throw new Error('Erreur serveur');
    const data = await response.json();
    setEvenements(data);
  } catch (err) {
    setError('Impossible de charger les événements.');
  } finally {
    setLoading(false);
  }
};

<FlatList
  data={evenements}
  keyExtractor={(item) => item.idEvent.toString()}
  renderItem={({ item }) => (
    <TouchableOpacity
      onPress={() => navigation.navigate('EventDetail', { eventId: item.idEvent })}
    >
      <Text>{item.titreEvent}</Text>
    </TouchableOpacity>
  )}
  ListEmptyComponent={<Text>Aucun événement</Text>}
/>
```

EventDetailScreen.js — Bouton Participer (TP7)

```
const getAuthData = async () => {
  const token = await AsyncStorage.getItem('token');
  const userId = await AsyncStorage.getItem('userId');
  return { token, userId };
};

const handleParticiper = async () => {
  setParticiperLoading(true);
  try {
    const { token, userId } = await getAuthData();
    if (!token || !userId) {
      Alert.alert('Non connecté', 'Veuillez vous connecter.',
        [{ text: 'Se connecter', onPress: () => navigation.replace('Login') }]);
```

```
      return;
    }
    const today = new Date().toISOString().split('T')[0];
    const response = await fetch(`${BASE_URL}/api/reservations/save`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${token}`,
      },
      body: JSON.stringify({
        dateReservation: today, nbrPersonnes: 1,
        user: { idUser: parseInt(userId) },
        evenement: { idEvent: eventId },
      }),
    });
    if (response.status === 200 || response.status === 201) {
      Alert.alert('Inscription confirmée !');
    } else if (response.status === 401) {
      await AsyncStorage.multiRemove(['token', 'userId']);
      navigation.replace('Login');
    } else if (response.status === 403) {
      Alert.alert('Accès refusé', 'Rôle ADMIN requis.');
    } else {
      const msg = await response.text();
      Alert.alert('Erreur', msg);
    }
  } catch {
    Alert.alert('Erreur réseau', 'Serveur inaccessible.');
  } finally {
    setParticiperLoading(false);
  }
};
```

Bouton avec spinner et désactivation pendant l'envoi :

```
<TouchableOpacity
  style={[styles.btn, participerLoading && styles.btnDisabled]}
  onPress={handleParticiper}
  disabled={participerLoading}
>
  {participerLoading
    ? <ActivityIndicator color='#fff' />
    : <Text style={styles.btnText}>Participer</Text>
  }
</TouchableOpacity>
```

FavoritesScreen.js

Bouton avec spinner et désactivation pendant l'envoi :

```
// ─── Écran principal ──────────────────────────────────────────────────────
export default function EventsListScreen({ navigation }) {
  const [events,     setEvents]     = useState([]);
  const [loading,    setLoading]    = useState(true);
  const [refreshing, setRefreshing] = useState(false);
  const [error,      setError]      = useState(null);
  const headerAnim = useRef(new Animated.Value(0)).current;

  useEffect(() => {
    // ✅ Initialiser la BDD SQLite au démarrage
    initFavorisDB();
    load();
    Animated.timing(headerAnim, { toValue: 1, duration: 550, useNativeDriver: true }).start();
  }, []);

  const load = async (isRef = false) => {
    if (isRef) setRefreshing(true); else setLoading(true);
```

```
  setError(null);
  try {
    const res = await fetch(`${BASE_URL}/api/evenements/all`);
    if (!res.ok) throw new Error(`Erreur serveur: ${res.status}`);
    const data = await res.json();
    setEvents(Array.isArray(data) ? data : []);
  } catch (e) {
    setError(`Impossible de charger les événements.\n${e.message}`);
  } finally { setLoading(false); setRefreshing(false); }
};

const Header = () => (
  <Animated.View style={[s.header, {
    opacity: headerAnim,
    transform: [{ translateY: headerAnim.interpolate({ inputRange:[0,1], outputRange:[-16,0] }) }],
  }]}>
    <Text style={s.hGreet}>Bonne journée 👋</Text>
    <Text style={s.hTitle}>Événements</Text>
    <View style={s.headerRow}>
      {events.length > 0 && (
        <View style={s.countBadge}>
          <Text style={s.countTxt}>{events.length} disponible{events.length > 1 ? 's' : ''}</Text>
        </View>
      )}
      {/* Bouton vers Mes Favoris */}
      <TouchableOpacity
        style={s.favBtn}
        onPress={() => navigation.navigate('Favorites')}
      >
        <Text style={s.favBtnTxt}>❤️  Mes favoris</Text>
      </TouchableOpacity>
    </View>
  </Animated.View>
);
```

6. Problèmes rencontrés et solutions

Expo Go ne se connecte pas — The request timed out
Cause : Le réseau scolaire bloque les connexions LAN directes entre le PC et le smartphone.
Solution : npx expo start --tunnel — les données transitent via les serveurs Expo.

```
npx expo start --tunnel
```

fetch() échoue — Network request failed
Cause : Sur smartphone, localhost pointe vers le téléphone lui-même, pas le PC.
Solution : Utiliser l'IP locale du PC dans services/api.js (ex: 192.168.1.42).

```
// ✗ — localhost = le téléphone
const BASE_URL = 'http://localhost:8081';

// ✓ — IP du PC sur le réseau Wi-Fi
const BASE_URL = 'http://192.168.1.42:8081';
```

AsyncStorage — erreur de type
Cause : AsyncStorage ne stocke que des strings — pas de nombres ni d'objets directement.
Solution : Convertir avec String() pour les nombres, JSON.stringify() pour les tableaux.

```
await AsyncStorage.setItem('userId', String(data.userId));
```

```
await AsyncStorage.setItem('roles', JSON.stringify(data.roles));
```

Retour possible vers le Login après connexion
Cause : navigate() empile les écrans — l'utilisateur peut revenir au Login avec le bouton retour.
Solution : Utiliser replace() qui remplace l'écran courant au lieu de l'empiler.

```
navigation.replace('EventsList'); // ✓ — remplace, pas de retour possible
```

Erreur 401 — token rejeté par le backend
Cause : Espace manquant après 'Bearer' dans le header Authorization.
Solution : Vérifier le format exact : 'Bearer ' + token (espace obligatoire).

```
// ✗
'Authorization': 'Bearer' + token,
// ✓
'Authorization': `Bearer ${token}`,
```

Erreur 403 — Accès refusé
Cause : Le compte connecté n'a pas le rôle ADMIN. L'endpoint POST /api/reservations/save requiert hasAuthority('ADMIN') dans SecurityConfig.java.
Solution : Vérifier / mettre à jour le rôle dans MySQL.

```
SELECT email, role FROM user WHERE email = 'votre@email.com';
UPDATE user SET role = 'ADMIN' WHERE email = 'votre@email.com';
```

useEffect déclenche des appels en boucle
Cause : Tableau de dépendances manquant ou mal configuré dans useEffect.
Solution : Utiliser un tableau vide [] pour un appel unique au montage de l'écran.

```
useEffect(() => {
  chargerEvenements();
}, []); // [] = une seule fois au montage
```

Titre de la barre de navigation pas mis à jour
Cause : Le titre est défini statiquement dans AppNavigator, pas mis à jour après le chargement des données.
Solution : Appeler navigation.setOptions() après avoir reçu les données de l'API.

```
const data = await response.json();
setEvenement(data);
navigation.setOptions({ title: data.titreEvent });
```

7. Tests et validation

Tests Postman — endpoints backend

Méthode	Endpoint	Auth	Code	Résultat
POST	/api/auth/login	Aucune	200	{ token, userId, roles }
GET	/api/evenements/all	Aucune (public)	200	Tableau JSON des événements
GET	/api/evenements/getById/1	Aucune (public)	200	Objet JSON événement ID 1

POST	/api/reservations/save	Bearer <token valide>	200	Réservation créée en BDD

Tests sur l'application (Expo Go)

LoginScreen
– Formulaire email/mot de passe affiché correctement
– Spinner visible pendant l'appel API
– Token stocké dans AsyncStorage après connexion réussie
– Navigation vers EventsList déclenchée automatiquement
– Pas de retour possible vers LoginScreen (replace)

EventsListScreen
– Événements chargés automatiquement au montage (useEffect)
– Chaque carte affiche : titre, type, date, ville, prix
– Bouton Réessayer visible si erreur réseau
– Clic sur une carte → navigation vers EventDetail avec l'eventId

EventDetailScreen
– Toutes les sections présentes : dates, description, lieu, tarif, places, organisateur
– Titre dynamique dans la barre de navigation
– Bouton Participer fixé en bas de l'écran (position: absolute)

Bouton Participer — Et Bouton Favorites
– ADMIN connecté → réservation créée → message confirmation ✓
– USER connecté → réservation créée → message confirmation ✓
– Token expiré → 401 → nettoyage AsyncStorage → retour Login ✓
– Pas connecté → message Non connecté + proposition login ✓
– Backend éteint → message Erreur réseau ✓
– Double-clic impossible : bouton désactivé pendant l'envoi ✓
– Réservation visible en MySQL : SELECT * FROM reservation; ✓

Bilan
Ce projet couvre l'ensemble du cycle d'une application mobile connectée : authentification JWT, consommation d'API REST, navigation entre écrans, persistance locale et requête POST sécurisée. L'application est testée sur appareil physique via Expo Go et fonctionne avec le backend Spring Boot.

Documentation Mobile
Projet Evenix-Mobile
React Native · Expo · JWT
BTS SIO SLAM 2026
Fatou Ndiaye